01 LC 206 | Reverse Linked List (Easy)

Problem: Given the head of a singly linked list, reverse the list, and return the reversed list.
Constraints: The number of nodes in the list is the range [0, 5000].; -5000 <= Node.val <= 5000

```
class Solution {
public:
  ListNode* reverseList(ListNode* head) {
    ListNode* dummy = new ListNode();
    ListNode* curr = head;
    while (curr) {
      ListNode* next = curr->next;
      curr->next = dummy->next;
      dummy->next = curr;
      curr = next;
    }
    return dummy->next;
  }
};
```

12 LC 876 | Middle of the Linked List (Easy)

Problem: Given the head of a singly linked list, return the middle node of the linked list. If there are two middle nodes, return the second middle node.
Constraints: The number of nodes in the list is in the range [1, 100].; 1 <= Node.val <= 100

```
class Solution {
public:
  ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
      slow = slow->next;
      fast = fast->next->next;
    }
    return slow;
  }
};
```

24 LC 237 | Delete Node in a Linked List (Medium)

Problem: Delete the given non-tail node from a singly linked list when only that node, not the head, is supplied. The resulting sequence must omit that value and preserve the others' order; physical deallocation is not required.
Constraints: The number of the nodes in the given list is in the range [2, 1000].; -1000 <= Node.val <= 1000; The value of each node in the list is unique.; The node to be deleted is in the list and is not a tail node.

```
class Solution {
public:
  void deleteNode(ListNode* node) {
    node->val = node->next->val;
    node->next = node->next->next;
  }
};
```

02 LC 21 | Merge Two Sorted Lists (Easy)

Problem: You are given the heads of two sorted linked lists list1 and list2. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.
Constraints: The number of nodes in both lists is in the range [0, 50].; -100 <= Node.val <= 100; Both list1 and list2 are sorted in non-decreasing order.

```
class Solution {
public:
  ListNode* mergeTwoLists(ListNode* list1,
    ListNode* list2) {
    ListNode* dummy = new ListNode();
    ListNode* curr = dummy;
    while (list1 && list2) {
      if (list1->val <= list2->val) {
        curr->next = list1;
        list1 = list1->next;
      } else {
        curr->next = list2;
        list2 = list2->next;
      }
      curr = curr->next;
    }
    curr->next = list1 ? list1 : list2;
    return dummy->next;
  }
};
```

06 LC 2 | Add Two Numbers (Medium)

Problem: You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.
Constraints: The number of nodes in each linked list is in the range [1, 100].; 0 <= Node.val <= 9; It is guaranteed that the list represents a number that does not have leading zeros.

```
class Solution {
public:
  ListNode* addTwoNumbers(ListNode* l1, ListNode*
    l2) {
    ListNode dummy;
    int carry = 0;
    ListNode* cur = &dummy;
    while (l1 || l2 || carry) {
      int s = (l1 ? l1->val : 0) + (l2 ? l2->val :
        0) + carry;
      carry = s / 10;
      cur->next = new ListNode(s % 10);
      cur = cur->next;
      l1 = l1 ? l1->next : nullptr;
      l2 = l2 ? l2->next : nullptr;
    }
    return dummy.next;
  }
};
```

03 LC 141 | Linked List Cycle (Easy)

Problem: Return whether following next pointers in the linked list eventually revisits a node. The judge's pos value describes a possible tail connection and is not a function argument.
Constraints: The number of the nodes in the list is in the range [0, 10^4].; -10^5 <= Node.val <= 10^5; pos is -1 or a valid index in the linked-list.

```
class Solution {
public:
  bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
      slow = slow->next;
      fast = fast->next->next;
      if (slow == fast) {
        return true;
      }
    }
    return false;
  }
};
```

11 LC 160 | Intersection of Two Linked Lists (Easy)

Problem: Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null. For example, the following two linked lists begin to intersect at node c1: The test cases are generated such that there are no cycles anywhere in the entire linked structure. Note that the linked lists must retain their original structure after the function returns.
Constraints: The number of nodes of listA is in the m.; The number of nodes of listB is in the n.; 1 <= m, n <= 3 * 10^4; 1 <= Node.val <= 10^5; 0 <= skipA <= m; 0 <= skipB <= n; intersectVal is 0 if listA and listB do not intersect.; intersectVal == listA[skipA] == listB[skipB] if listA and listB intersect.

```
class Solution {
public:
  ListNode* getIntersectionNode(ListNode* headA,
    ListNode* headB) {
    ListNode *a = headA, *b = headB;
    while (a != b) {
      a = a ? a->next : headB;
      b = b ? b->next : headA;
    }
    return a;
  }
};
```

21 LC 147 | Insertion Sort List (Medium)

Problem: Sort the linked list in ascending order using insertion sort and return its head. Move nodes into their proper positions in a growing sorted prefix.
Constraints: The number of nodes in the list is in the range [1, 5000].; -5000 <= Node.val <= 5000

```
class Solution {
public:
  ListNode* insertionSortList(ListNode* head) {
    ListNode dummy(0, head);
    ListNode* tail = head;
    while (tail && tail->next) {
      ListNode* node = tail->next;
      if (tail->val <= node->val) { tail = node;
        continue; }
      ListNode* before = &dummy;
      while (before->next->val <= node->val)
        before = before->next;
      tail->next = node->next;
      node->next = before->next;
      before->next = node;
    }
    return dummy.next;
  }
};
```

04 LC 142 | Linked List Cycle II (Medium)

Problem: Return the node where the linked list's cycle begins, or null if acyclic. Do not modify the list. The judge's pos value is not supplied as an argument.
Constraints: The number of the nodes in the list is in the range [0, 10^4].; -10^5 <= Node.val <= 10^5; pos is -1 or a valid index in the linked-list.

```
class Solution {
public:
  ListNode* detectCycle(ListNode* head) {
    ListNode* fast = head;
    ListNode* slow = head;
    while (fast && fast->next) {
      slow = slow->next;
      fast = fast->next->next;
      if (slow == fast) {
        ListNode* ans = head;
        while (ans != slow) {
          ans = ans->next;
          slow = slow->next;
        }
        return ans;
      }
    }
    return nullptr;
  }
};
```

26 LC 430 | Flatten a Multilevel Doubly Linked List (Medium)

Problem: Flatten a multilevel doubly linked list in depth-first order: a node's child list appears immediately after that node, before its old next node. Reuse nodes, repair prev/next links, clear every child pointer, and return the head.
Constraints: The number of Nodes will not exceed 1000.; 1 <= Node.val <= 10^5 How the multilevel linked list is represented in test cases: We use the multilevel linked list from Example 1 above: 1---2---3---4---5---6--NULL | 7---8---9---10--NULL | 11--12--NULL The serialization of each level is as follows: [1,2,3,4,5,6,null] [7,8,9,10,null] [11,12,null] To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes: [1, 2, 3, 4, 5, 6, null] | [null, null, 7, 8, 9, 10, null] | [null, 11, 12, null] Merging the serialization of each level and removing trailing nulls we obtain: [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

```
class Solution {
public:
  Node* flatten(Node* head) {
    flattenGetTail(head);
    return head;
  }
  Node* flattenGetTail(Node* head) {
    Node* cur = head;
    Node* tail = nullptr;
    while (cur) {
      Node* next = cur->next;
      if (cur->child) {
        Node* child = cur->child;
        Node* childTail =
          flattenGetTail(cur->child);
        cur->child = nullptr;
        cur->next = child;
        child->prev = cur;
        childTail->next = next;
        if (next)
          next->prev = childTail;
        tail = childTail;
      } else {
        tail = cur;
      }
      cur = next;
    }
    return tail;
  }
};
```

05 LC 19 | Remove Nth Node From End of List (Medium)

Problem: Given the head of a linked list, remove the nth node from the end of the list and return its head.

Constraints: The number of nodes in the list is sz.; 1 <= sz <= 30; 0 <= Node.val <= 100; 1 <= n <= sz

```
class Solution {
public:
  ListNode* removeNthFromEnd(ListNode* head, int
    n) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* fast = dummy;
    ListNode* slow = dummy;
    while (n--) {
      fast = fast->next;
    }
    while (fast->next) {
      slow = slow->next;
      fast = fast->next;
    }
    slow->next = slow->next->next;
    return dummy->next;
  }
};
```

25 LC 328 | Odd Even Linked List (Medium)

Problem: Rearrange a linked list so nodes at odd one-based positions precede nodes at even positions, preserving order within both groups. Use O(n) time and O(1) extra space.

Constraints: The number of nodes in the linked list is in the range [0, 10^4].; -10^6 <= Node.val <= 10^6

```
class Solution {
public:
  ListNode* oddEvenList(ListNode* head) {
    if (!head) {
      return nullptr;
    }
    ListNode* a = head;
    ListNode *b = head->next, *c = b;
    while (b && b->next) {
      a->next = b->next;
      a = a->next;
      b->next = a->next;
      b = b->next;
    }
    a->next = c;
    return head;
  }
};
```

45 LC 114 | Flatten Binary Tree to Linked List (Medium)

Problem: Given the root of a binary tree, flatten the tree into a "linked list": - The "linked list" should use the same TreeNode class where the right child pointer points to the next node in the list and the left child pointer is always null. - The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Constraints: The number of nodes in the tree is in the range [0, 2000].; -100 <= Node.val <= 100

```
class Solution {
public:
  void flatten(TreeNode* root) {
    while (root) {
      if (root->left) {
        TreeNode* pre = root->left;
        while (pre->right) {
          pre = pre->right;
        }
        pre->right = root->right;
        root->right = root->left;
        root->left = nullptr;
      }
      root = root->right;
    }
  }
};
```

07 LC 23 | Merge k Sorted Lists (Hard)

Problem: You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Constraints: k == lists.length; 0 <= k <= 10^4; 0 <= lists[i].length <= 500; -10^4 <= lists[i][j] <= 10^4; lists[i] is sorted in ascending order.; The sum of lists[i].length will not exceed 10^4.

```
class Solution {
public:
  ListNode* mergeKLists(vector<ListNode*>& lists)
    {
    auto cmp = [](ListNode* a, ListNode* b) {
      return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>,
      decltype(cmp)> pq;
    for (auto head : lists) {
      if (head) {
        pq.push(head);
      }
    }
    ListNode* dummy = new ListNode();
    ListNode* cur = dummy;
    while (!pq.empty()) {
      ListNode* node = pq.top();
      pq.pop();
      if (node->next) {
        pq.push(node->next);
      }
      cur->next = node;
      cur = cur->next;
    }
    return dummy->next;
  }
};
```

35 LC 2130 | Maximum Twin Sum of a Linked List (Medium)

Problem: The list has even length n. Node i is paired with node n-1-i. Return the maximum sum of a twin pair.

Constraints: The number of nodes in the list is an even integer in the range [2, 10^5].; 1 <= Node.val <= 10^5

```
class Solution {
public:
  int pairSum(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head->next;
    while (fast && fast->next) {
      slow = slow->next;
      fast = fast->next->next;
    }
    ListNode* pa = head;
    ListNode* q = slow->next;
    slow->next = nullptr;
    ListNode* pb = reverse(q);
    int ans = 0;
    while (pa) {
      ans = max(ans, pa->val + pb->val);
      pa = pa->next;
      pb = pb->next;
    }
    return ans;
  }
  ListNode* reverse(ListNode* head) {
    ListNode* dummy = new ListNode();
    ListNode* curr = head;
    while (curr) {
      ListNode* next = curr->next;
      curr->next = dummy->next;
      dummy->next = curr;
      curr = next;
    }
    return dummy->next;
  }
};
```

08 LC 25 | Reverse Nodes in k-Group (Hard)

Problem: Reverse linked-list nodes in consecutive groups of k. Leave the final group unchanged if it has fewer than k nodes. Change links, not node values; use O(1) extra space.

Constraints: The number of nodes in the list is n.; 1 <= k <= n <= 5000; 0 <= Node.val <= 1000

```
class Solution {
public:
  ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* pre = dummy;
    while (pre != nullptr) {
      ListNode* cur = pre;
      for (int i = 0; i < k; i++) {
        cur = cur->next;
        if (cur == nullptr) {
          return dummy->next;
        }
      }
      ListNode* node = pre->next;
      ListNode* nxt = cur->next;
      cur->next = nullptr;
      pre->next = reverse(node);
      node->next = nxt;
      pre = node;
    }
    return dummy->next;
  }
private:
  ListNode* reverse(ListNode* head) {
    ListNode* dummy = new ListNode();
    ListNode* cur = head;
    while (cur != nullptr) {
      ListNode* nxt = cur->next;
      cur->next = dummy->next;
      dummy->next = cur;
      cur = nxt;
    }
    return dummy->next;
  }
};
```

28 LC 725 | Split Linked List in Parts (Medium)

Problem: Split the list into k consecutive parts. Part sizes differ by at most one, earlier parts are never smaller, and empty parts are allowed. Return their heads in original order.

Constraints: The number of nodes in the list is in the range [0, 1000].; 0 <= Node.val <= 1000; 1 <= k <= 50

```
class Solution {
public:
  vector<ListNode*> splitListToParts(ListNode*
    head, int k) {
    int n = 0;
    for (ListNode* cur = head; cur != nullptr; cur
      = cur->next) {
      ++n;
    }
    int cnt = n / k, mod = n % k;
    vector<ListNode*> ans(k, nullptr);
    ListNode* cur = head;
    for (int i = 0; i < k && cur != nullptr; ++i)
      {
      ans[i] = cur;
      int m = cnt + (i < mod ? 1 : 0);
      for (int j = 1; j < m; ++j) {
        cur = cur->next;
      }
      ListNode* nxt = cur->next;
      cur->next = nullptr;
      cur = nxt;
    }
    return ans;
  }
};
```

09 LC 143 | Reorder List (Medium)

Problem: You are given the head of a singly linked-list. The list can be represented as: L_0 → L_1 → … → L_n - 1 → L_n Reorder the list to be on the following form: L_0 → L_n → L_1 → L_n - 1 → L_2 → L_n - 2 → … You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Constraints: The number of nodes in the list is in the range [1, 5 * 10^4].; 1 <= Node.val <= 1000

```
class Solution {
public:
  void reorderList(ListNode* head) {
    ListNode* fast = head;
    ListNode* slow = head;
    while (fast->next && fast->next->next) {
      slow = slow->next;
      fast = fast->next->next;
    }
    ListNode* cur = slow->next;
    slow->next = nullptr;
    ListNode* pre = nullptr;
    while (cur) {
      ListNode* t = cur->next;
      cur->next = pre;
      pre = cur;
      cur = t;
    }
    cur = head;
    while (pre) {
      ListNode* t = pre->next;
      pre->next = cur->next;
      cur->next = pre;
      cur = pre->next;
      pre = t;
    }
  }
};
```

17 LC 61 | Rotate List (Medium)

Problem: Given the head of a linked list, rotate the list to the right by k places.

Constraints: The number of nodes in the list is in the range [0, 500].; -100 <= Node.val <= 100; 0 <= k <= 2 * 10^9

```
class Solution {
public:
  ListNode* rotateRight(ListNode* head, int k) {
    if (!head || !head->next) {
      return head;
    }
    ListNode* cur = head;
    int n = 0;
    while (cur) {
      ++n;
      cur = cur->next;
    }
    k %= n;
    if (k == 0) {
      return head;
    }
    ListNode* fast = head;
    ListNode* slow = head;
    while (k--) {
      fast = fast->next;
    }
    while (fast->next) {
      fast = fast->next;
      slow = slow->next;
    }
    ListNode* ans = slow->next;
    slow->next = nullptr;
    fast->next = head;
    return ans;
  }
};
```

10 LC 234 | Palindrome Linked List (Easy)
Problem: Given the head of a singly linked list, return true if it is a palindrome or false otherwise.
Constraints: The number of nodes in the list is in the range [1, 10^5].; 0 <= Node.val <= 9

```
class Solution {
public:
  bool isPalindrome(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head->next;
    while (fast && fast->next) {
      slow = slow->next;
      fast = fast->next->next;
    }
    ListNode* pre = nullptr;
    ListNode* cur = slow->next;
    while (cur) {
      ListNode* t = cur->next;
      cur->next = pre;
      pre = cur;
      cur = t;
    }
    while (pre) {
      if (pre->val != head->val) return false;
      pre = pre->next;
      head = head->next;
    }
    return true;
  }
};
```

27 LC 707 | Design Linked List (Medium)
Problem: Implement MyLinkedList with zero-based get(index), addAtHead(val), addAtTail(val), addAtIndex(index,val), deleteAtIndex(index). get returns -1 for invalid indices; invalid deletions do nothing. Insert before index, append at size, and ignore indices beyond size; nonpositive insertion indices mean the head.
Constraints: 0 <= index, val <= 1000; Please do not use the built-in LinkedList library.; At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

```
class MyLinkedList {
  list<int> nodes;
  list<int>::iterator locate(int index) {
    if (index <= static_cast<int>(nodes.size()) /
      2) return next(nodes.begin(), index);
    return prev(nodes.end(),
      static_cast<int>(nodes.size()) - index);
  }
public:
  MyLinkedList() = default;
  int get(int index) {
    if (index < 0 || index >=
      static_cast<int>(nodes.size())) return -1;
    return *locate(index);
  }
  void addAtHead(int val) { nodes.push_front(val);
    }
  void addAtTail(int val) { nodes.push_back(val);
    }
  void addAtIndex(int index, int val) {
    index = max(0, index);
    if (index > static_cast<int>(nodes.size()))
      return;
    nodes.insert(locate(index), val);
  }
  void deleteAtIndex(int index) {
    if (index < 0 || index >=
      static_cast<int>(nodes.size())) return;
    nodes.erase(locate(index));
  }
};
```

13 LC 83 | Remove Duplicates from Sorted List (Easy)
Problem: Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.
Constraints: The number of nodes in the list is in the range [0, 300].; -100 <= Node.val <= 100; The list is guaranteed to be sorted in ascending order.

```
class Solution {
public:
  ListNode* deleteDuplicates(ListNode* head) {
    ListNode* cur = head;
    while (cur != nullptr && cur->next != nullptr)
      {
      if (cur->val == cur->next->val) {
        cur->next = cur->next->next;
      } else {
        cur = cur->next;
      }
    }
    return head;
  }
};
```

32 LC 1721 | Swapping Nodes in a Linked List (Medium)
Problem: You are given the head of a linked list, and an integer k. Return the head of the linked list after swapping the values of the kth node from the beginning and the kth node from the end (the list is 1-indexed).
Constraints: The number of nodes in the list is n.; 1 <= k <= n <= 10^5; 0 <= Node.val <= 100

```
class Solution {
public:
  ListNode* swapNodes(ListNode* head, int k) {
    ListNode* fast = head;
    while (--k) {
      fast = fast->next;
    }
    ListNode* slow = head;
    ListNode* p = fast;
    while (fast->next) {
      fast = fast->next;
      slow = slow->next;
    }
    ListNode* q = slow;
    swap(p->val, q->val);
    return head;
  }
};
```

41 LC 369 | Plus One Linked List (Medium)
Problem: Given a non-negative integer represented as a linked list of digits, plus one to the integer. The digits are stored such that the most significant digit is at the head of the list.
Constraints: The number of nodes in the linked list is in the range [1, 100].; 0 <= Node.val <= 9; The number represented by the linked list does not contain leading zeros except for the zero itself.

```
class Solution {
public:
  ListNode* plusOne(ListNode* head) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* target = dummy;
    for (; head; head = head->next) {
      if (head->val != 9) {
        target = head;
      }
    }
    target->val++;
    for (target = target->next; target; target =
      target->next) {
      target->val = 0;
    }
    return dummy->val ? dummy : dummy->next;
  }
};
```

14 LC 82 | Remove Duplicates from Sorted List II (Medium)

Problem: You are given the head of a sorted linked list. Delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list. Return the linked list sorted as well.
Constraints: The number of nodes in the list is in the range [0, 300].; -100 <= Node.val <= 100; The list is guaranteed to be sorted in ascending order.

```
class Solution {
public:
  ListNode* deleteDuplicates(ListNode* head) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* pre = dummy;
    ListNode* cur = head;
    while (cur) {
      while (cur->next && cur->next->val ==
        cur->val) {
        cur = cur->next;
      }
      if (pre->next == cur) {
        pre = cur;
      } else {
        pre->next = cur->next;
      }
      cur = cur->next;
    }
    return dummy->next;
  }
};
```

18 LC 86 | Partition List (Medium)

Problem: Given the head of a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x. You should preserve the original relative order of the nodes in each of the two partitions.
Constraints: The number of nodes in the list is in the range [0, 200].; -100 <= Node.val <= 100; -200 <= x <= 200

```
class Solution {
public:
  ListNode* partition(ListNode* head, int x) {
    ListNode* l = new ListNode();
    ListNode* r = new ListNode();
    ListNode* tl = l;
    ListNode* tr = r;
    for (; head; head = head->next) {
      if (head->val < x) {
        tl->next = head;
        tl = tl->next;
      } else {
        tr->next = head;
        tr = tr->next;
      }
    }
    tr->next = nullptr;
    tl->next = r->next;
    return l->next;
  }
};
```

15 LC 92 | Reverse Linked List II (Medium)

Problem: Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.
Constraints: The number of nodes in the list is n.; 1 <= n <= 500; -500 <= Node.val <= 500; 1 <= left <= right <= n

```
class Solution {
public:
  ListNode* reverseBetween(ListNode* head, int
    left, int right) {
    if (!head->next || left == right) {
      return head;
    }
    ListNode* dummy = new ListNode(0, head);
    ListNode* pre = dummy;
    for (int i = 0; i < left - 1; ++i) {
      pre = pre->next;
    }
    ListNode *p = pre, *q = pre->next;
    ListNode* cur = q;
    for (int i = 0; i < right - left + 1; ++i) {
      ListNode* t = cur->next;
      cur->next = pre;
      pre = cur;
      cur = t;
    }
    p->next = pre;
    q->next = cur;
    return dummy->next;
  }
};
```

22 LC 148 | Sort List (Medium)

Problem: Given the head of a linked list, return the list after sorting it in ascending order.
Constraints: The number of nodes in the list is in the range [0, 5 * 10^4].; -10^5 <= Node.val <= 10^5

```
class Solution {
  ListNode* split(ListNode* head, int size) {
    if (!head) return nullptr;
    for (int i = 1; i < size && head->next; ++i)
      head = head->next;
    ListNode* rest = head->next;
    head->next = nullptr;
    return rest;
  }
public:
  ListNode* sortList(ListNode* head) {
    int n = 0;
    for (auto p = head; p; p = p->next) ++n;
    ListNode dummy(0, head);
    for (int size = 1; size < n; size *= 2) {
      ListNode* current = dummy.next;
      ListNode* tail = &dummy;
      while (current) {
        ListNode* a = current;
        ListNode* b = split(a, size);
        current = split(b, size);
        while (a && b) {
          ListNode*& smaller = a->val <= b->val ?
            a : b;
          tail->next = smaller;
          smaller = smaller->next;
          tail = tail->next;
        }
        tail->next = a ? a : b;
        while (tail->next) tail = tail->next;
      }
    }
    return dummy.next;
  }
};
```

16 LC 24 | Swap Nodes in Pairs (Medium)

Problem: Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Constraints: The number of nodes in the list is in the range [0, 100].; 0 <= Node.val <= 100

```
class Solution {
public:
  ListNode* swapPairs(ListNode* head) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* pre = dummy;
    ListNode* cur = head;
    while (cur && cur->next) {
      ListNode* t = cur->next;
      cur->next = t->next;
      t->next = cur;
      pre->next = t;
      pre = cur;
      cur = cur->next;
    }
    return dummy->next;
  }
};
```

48 LC 622 | Design Circular Queue (Medium)

Problem: Implement MyCircularQueue(k): enQueue(value)/deQueue() report success; insertions fail when full and deletions when empty. Front()/Rear() return end values or -1 when empty. isEmpty()/isFull() report state. Use circular storage; do not use a built-in queue.

Constraints: 1 <= k <= 1000; 0 <= value <= 1000; At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.

```
class MyCircularQueue {
private:
  int front;
  int size;
  int capacity;
  vector<int> q;
public:
  MyCircularQueue(int k) {
    capacity = k;
    q = vector<int>(k);
    front = size = 0;
  }
  bool enQueue(int value) {
    if (isFull()) return false;
    int idx = (front + size) % capacity;
    q[idx] = value;
    ++size;
    return true;
  }
  bool deQueue() {
    if (isEmpty()) return false;
    front = (front + 1) % capacity;
    --size;
    return true;
  }
  int Front() {
    if (isEmpty()) return -1;
    return q[front];
  }
  int Rear() {
    if (isEmpty()) return -1;
    int idx = (front + size - 1) % capacity;
    return q[idx];
  }
  bool isEmpty() {
    return size == 0;
  }
  bool isFull() {
    return size == capacity;
  }
};
```

19 LC 138 | Copy List with Random Pointer (Medium)

Problem: Deep-copy a list whose nodes have next and random pointers. random points to any original node or null. Every copied pointer must reference the corresponding copied node, never an original node; return the copied head.

Constraints: 0 <= n <= 1000; -10^4 <= Node.val <= 10^4; Node.random is null or is pointing to some node in the linked list.

```
class Solution {
public:
  Node* copyRandomList(Node* head) {
    if (!head) {
      return nullptr;
    }
    Node* cur = head;
    while (cur != nullptr) {
      Node* node = new Node(cur->val);
      node->next = cur->next;
      cur->next = node;
      cur = node->next;
    }
    cur = head;
    while (cur != nullptr) {
      cur->next->random = cur->random == nullptr ?
        nullptr : cur->random->next;
      cur = cur->next->next;
    }
    cur = head;
    Node* ans = head->next;
    while (cur->next != nullptr) {
      Node* node = cur->next;
      cur->next = node->next;
      cur = node;
    }
    return ans;
  }
};
```

40 LC 445 | Add Two Numbers II (Medium)

Problem: You are given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Constraints: The number of nodes in each linked list is in the range [1, 100].; 0 <= Node.val <= 9; It is guaranteed that the list represents a number that does not have leading zeros.

```
class Solution {
public:
  ListNode* addTwoNumbers(ListNode* l1, ListNode*
    l2) {
    stack<int> s1;
    stack<int> s2;
    for (; l1; l1 = l1->next) s1.push(l1->val);
    for (; l2; l2 = l2->next) s2.push(l2->val);
    ListNode* dummy = new ListNode();
    int carry = 0;
    while (!s1.empty() || !s2.empty() || carry) {
      int s = carry;
      if (!s1.empty()) {
        s += s1.top();
        s1.pop();
      }
      if (!s2.empty()) {
        s += s2.top();
        s2.pop();
      }
      dummy->next = new ListNode(s % 10,
        dummy->next);
      carry = s / 10;
    }
    return dummy->next;
  }
};
```

20 LC 146 | LRU Cache (Medium)
Problem: Implement LRUCache(capacity): get(key) returns its value or -1; put(key,value) inserts or updates. When insertion exceeds capacity, evict the least recently used entry. Both operations require average O(1) time.
Constraints: 1 <= capacity <= 3000; 0 <= key <= 10^4; 0 <= value <= 10^5; At most 2 * 10^5 calls will be made to get and put.

```
class LRUCache {
  size_t capacity;
  list<pair<int, int>> entries;
  unordered_map<int, list<pair<int,
    int>>::iterator> location;
public:
  LRUCache(int capacity) : capacity(capacity) {}
  LRUCache(const LRUCache&) = delete;
  LRUCache& operator=(const LRUCache&) = delete;
  int get(int key) {
    auto it = location.find(key);
    if (it == location.end()) return -1;
    entries.splice(entries.begin(), entries,
      it->second);
    return it->second->second;
  }
  void put(int key, int value) {
    if (auto it = location.find(key); it !=
      location.end()) {
      it->second->second = value;
      entries.splice(entries.begin(), entries,
        it->second);
      return;
    }
    if (capacity == 0) return;
    if (entries.size() == capacity) {
      location.erase(entries.back().first);
      entries.pop_back();
    }
    entries.emplace_front(key, value);
    location[key] = entries.begin();
  }
};
```

31 LC 1171 | Remove Zero Sum Consecutive Nodes from Linked List (Medium)
Problem: Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences. After doing so, return the head of the final linked list. You may return any such answer. (Note that in the examples below, all sequences are serializations of ListNode objects.)
Constraints: The given linked list will contain between 1 and 1000 nodes.; Each node in the linked list has -1000 <= node.val <= 1000.

```
class Solution {
public:
  ListNode* removeZeroSumSublists(ListNode* head)
    {
    ListNode* dummy = new ListNode(0, head);
    unordered_map<int, ListNode*> last;
    ListNode* cur = dummy;
    int s = 0;
    while (cur) {
      s += cur->val;
      last[s] = cur;
      cur = cur->next;
    }
    s = 0;
    cur = dummy;
    while (cur) {
      s += cur->val;
      cur->next = last[s]->next;
      cur = cur->next;
    }
    return dummy->next;
  }
};
```

23 LC 203 | Remove Linked List Elements (Easy)
Problem: Given the head of a linked list and an integer val, remove all the nodes of the linked list that has Node.val == val, and return the new head.
Constraints: The number of nodes in the list is in the range [0, 10^4].; 1 <= Node.val <= 50; 0 <= val <= 50

```
class Solution {
public:
  ListNode* removeElements(ListNode* head, int
    val) {
    ListNode* dummy = new ListNode();
    dummy->next = head;
    ListNode* p = dummy;
    while (p->next) {
      if (p->next->val == val) {
        p->next = p->next->next;
      } else {
        p = p->next;
      }
    }
    return dummy->next;
  }
};
```

39 LC 2807 | Insert Greatest Common Divisors in Linked List (Medium)
Problem: Given the head of a linked list head, in which each node contains an integer value. Between every pair of adjacent nodes, insert a new node with a value equal to the greatest common divisor of them. Return the linked list after insertion. The greatest common divisor of two numbers is the largest positive integer that evenly divides both numbers.
Constraints: The number of nodes in the list is in the range [1, 5000].; 1 <= Node.val <= 1000

```
class Solution {
public:
  ListNode* insertGreatestCommonDivisors(ListNode*
    head) {
    ListNode* pre = head;
    for (ListNode* cur = head->next; cur; cur =
      cur->next) {
      int x = gcd(pre->val, cur->val);
      pre->next = new ListNode(x, cur);
      pre = cur;
    }
    return head;
  }
};
```

44 LC 109 | Convert Sorted List to Binary Search Tree (Medium)
Problem: Given the head of a singly linked list where elements are sorted in ascending order, convert it to a height-balanced binary search tree.
Constraints: The number of nodes in head is in the range [0, 2 * 10^4].; -10^5 <= Node.val <= 10^5

```
class Solution {
  TreeNode* build(ListNode*& cursor, int count) {
    if (!count) return nullptr;
    TreeNode* left = build(cursor, count / 2);
    TreeNode* root = new TreeNode(cursor->val);
    cursor = cursor->next;
    root->left = left;
    root->right = build(cursor, count - count / 2
      - 1);
    return root;
  }
public:
  TreeNode* sortedListToBST(ListNode* head) {
    int n = 0;
    for (auto p = head; p; p = p->next) ++n;
    return build(head, n);
  }
};
```

29 LC 817 | Linked List Components (Medium)

Problem: You are given the head of a linked list containing unique integer values and an integer array nums that is a subset of the linked list values. Return the number of connected components in nums. A connected component is a non-empty, maximal sequence of consecutive nodes in the linked list such that every node's value belongs to nums.

Constraints: The number of nodes in the linked list is n.; 1 <= n <= 10^4; 0 <= Node.val < n; All the values Node.val are unique.; 1 <= nums.length <= n; 0 <= nums[i] < n; All the values of nums are unique.

```
class Solution {
public:
  int numComponents(ListNode* head, vector<int>&
    nums) {
    unordered_set<int> s(nums.begin(),
      nums.end());
    int ans = 0;
    while (head) {
      while (head && !s.count(head->val)) head =
        head->next;
      ans += head != nullptr;
      while (head && s.count(head->val)) head =
        head->next;
    }
    return ans;
  }
};
```

33 LC 1669 | Merge In Between Linked Lists (Medium)

Problem: You are given two linked lists: list1 and list2 of sizes n and m respectively. Remove list1's nodes from the ath node to the bth node, and put list2 in their place. The blue edges and nodes in the following figure indicate the result: Build the result list and return its head.

Constraints: 3 <= list1.length <= 10^4; 1 <= a <= b < list1.length - 1; 1 <= list2.length <= 10^4

```
class Solution {
public:
  ListNode* mergeInBetween(ListNode* list1, int a,
    int b, ListNode* list2) {
    auto p = list1, q = list1;
    while (--a) {
      p = p->next;
    }
    while (b--) {
      q = q->next;
    }
    p->next = list2;
    while (p->next) {
      p = p->next;
    }
    p->next = q->next;
    q->next = nullptr;
    return list1;
  }
};
```

30 LC 1019 | Next Greater Node In Linked List (Medium)

Problem: For each linked-list node, return the first strictly greater value appearing later in the list, or 0 if none exists. Return answers in node order.

Constraints: The number of nodes in the list is n.; 1 <= n <= 10^4; 1 <= Node.val <= 10^9

```
class Solution {
public:
  vector<int> nextLargerNodes(ListNode* head) {
    vector<int> nums;
    for (; head; head = head->next) {
      nums.push_back(head->val);
    }
    stack<int> stk;
    int n = nums.size();
    vector<int> ans(n);
    for (int i = n - 1; ~i; --i) {
      while (!stk.empty() && stk.top() <= nums[i])
        {
        stk.pop();
      }
      if (!stk.empty()) {
        ans[i] = stk.top();
      }
      stk.push(nums[i]);
    }
    return ans;
  }
};
```

42 LC 708 | Insert into a Sorted Circular Linked List (Medium)

Problem: Insert insertVal into a nondecreasing circular linked list, preserving circular sorted order. The supplied node need not be the smallest. Return the original node, or the new self-linked node if the list was empty.

Constraints: The number of nodes in the list is in the range [0, 5 * 10^4].; -10^6 <= Node.val, insertVal <= 10^6

```
class Solution {
public:
  Node* insert(Node* head, int insertVal) {
    Node* node = new Node(insertVal);
    if (!head) {
      node->next = node;
      return node;
    }
    Node *prev = head, *curr = head->next;
    while (curr != head) {
      if ((prev->val <= insertVal && insertVal <=
        curr->val) || (prev->val > curr->val &&
        (insertVal >= prev->val || insertVal <=
        curr->val))) break;
      prev = curr;
      curr = curr->next;
    }
    prev->next = node;
    node->next = curr;
    return head;
  }
};
```

34 LC 2095 | Delete the Middle Node of a Linked List (Medium)
Problem: Delete the node at zero-based index floor(n/2) from the linked list and return the head. Thus an even-length list loses its second middle node; a one-node list becomes empty.
Constraints: The number of nodes in the list is in the range [1, 10^5].; 1 <= Node.val <= 10^5

```
class Solution {
public:
  ListNode* deleteMiddle(ListNode* head) {
    ListNode* dummy = new ListNode(0, head);
    ListNode* slow = dummy;
    ListNode* fast = head;
    while (fast && fast->next) {
      slow = slow->next;
      fast = fast->next->next;
    }
    slow->next = slow->next->next;
    return dummy->next;
  }
};
```

36 LC 2181 | Merge Nodes in Between Zeros (Medium)
Problem: You are given the head of a linked list, which contains a series of integers separated by 0's. The beginning and end of the linked list will have Node.val == 0. For every two consecutive 0's, merge all the nodes lying in between them into a single node whose value is the sum of all the merged nodes. The modified list should not contain any 0's. Return the head of the modified linked list.
Constraints: The number of nodes in the list is in the range [3, 2 * 10^5].; 0 <= Node.val <= 1000; There are no two consecutive nodes with Node.val == 0.; The beginning and end of the linked list have Node.val == 0.

```
class Solution {
public:
  ListNode* mergeNodes(ListNode* head) {
    ListNode* write = head;
    ListNode* tail = nullptr;
    int sum = 0;
    for (auto p = head->next; p; p = p->next) {
      if (p->val) sum += p->val;
      else { write->val = sum; tail = write; write
        = write->next; sum = 0; }
    }
    tail->next = nullptr;
    return head;
  }
};
```

46 LC 116 | Populating Next Right Pointers in Each Node (Medium)
Problem: You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition: struct Node { int val; Node *left; Node *right; Node *next; } Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL. Initially, all next pointers are set to NULL.
Constraints: The number of nodes in the tree is in the range [0, 2^12 - 1].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  Node* connect(Node* root) {
    if (!root) return root;
    root->next = nullptr;
    for (Node* level = root; level->left; level =
      level->left) {
      for (Node* p = level; p; p = p->next) {
        p->left->next = p->right;
        p->right->next = p->next ? p->next->left :
          nullptr;
      }
    }
    return root;
  }
};
```

37 LC 2487 | Remove Nodes From Linked List (Medium)
Problem: You are given the head of a linked list. Remove every node which has a node with a greater value anywhere to the right side of it. Return the head of the modified linked list.
Constraints: The number of the nodes in the given list is in the range [1, 10^5].; 1 <= Node.val <= 10^5

```
class Solution {
public:
  ListNode* removeNodes(ListNode* head) {
    ListNode* reversed = nullptr;
    while (head) {
      auto next = head->next;
      head->next = reversed;
      reversed = head;
      head = next;
    }
    ListNode* result = nullptr;
    int maximum = INT_MIN;
    while (reversed) {
      auto next = reversed->next;
      if (reversed->val >= maximum) {
        maximum = reversed->val;
        reversed->next = result;
        result = reversed;
      }
      reversed = next;
    }
    return result;
  }
};
```

38 LC 2816 | Double a Number Represented as a Linked List (Medium)
Problem: You are given the head of a non-empty linked list representing a non-negative integer without leading zeroes. Return the head of the linked list after doubling it.
Constraints: The number of nodes in the list is in the range [1, 10^4]; 0 <= Node.val <= 9; The input is generated such that the list represents a number that does not have leading zeros, except the number 0 itself.

```
class Solution {
public:
  ListNode* doubleIt(ListNode* head) {
    head = reverse(head);
    ListNode* dummy = new ListNode();
    ListNode* cur = dummy;
    int mul = 2, carry = 0;
    while (head) {
      int x = head->val * mul + carry;
      carry = x / 10;
      cur->next = new ListNode(x % 10);
      cur = cur->next;
      head = head->next;
    }
    if (carry) {
      cur->next = new ListNode(carry);
    }
    return reverse(dummy->next);
  }
  ListNode* reverse(ListNode* head) {
    ListNode* dummy = new ListNode();
    ListNode* cur = head;
    while (cur) {
      ListNode* next = cur->next;
      cur->next = dummy->next;
      dummy->next = cur;
      cur = next;
    }
    return dummy->next;
  }
};
```

43 LC 426 | Convert Binary Search Tree to Sorted Doubly Linked List (Medium)

Problem: Convert a BST in place into a sorted circular doubly linked list. Reuse left as previous and right as next, connect the smallest/largest nodes circularly, and return the smallest node, or null for an empty tree.
Constraints: The number of nodes in the tree is in the range [0, 2000].; -1000 <= Node.val <= 1000; All the values of the tree are unique.

```
class Solution {
public:
  Node* prev;
  Node* head;
  Node* treeToDoublyList(Node* root) {
    if (!root) return nullptr;
    prev = nullptr;
    head = nullptr;
    dfs(root);
    prev->right = head;
    head->left = prev;
    return head;
  }
  void dfs(Node* root) {
    if (!root) return;
    dfs(root->left);
    if (prev) {
      prev->right = root;
      root->left = prev;
    } else
      head = root;
    prev = root;
    dfs(root->right);
  }
};
```

47 LC 117 | Populating Next Right Pointers in Each Node II (Medium)

Problem: Given a binary tree struct Node { int val; Node *left; Node *right; Node *next; } Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL. Initially, all next pointers are set to NULL.
Constraints: The number of nodes in the tree is in the range [0, 6000].; -100 <= Node.val <= 100

```
class Solution {
public:
  Node* connect(Node* root) {
    Node* node = root;
    Node* prev = nullptr;
    Node* next = nullptr;
    auto modify = [&](Node* curr) {
      if (!curr) {
        return;
      }
      if (!next) {
        next = curr;
      }
      if (prev) {
        prev->next = curr;
      }
      prev = curr;
    };
    while (node) {
      prev = next = nullptr;
      while (node) {
        modify(node->left);
        modify(node->right);
        node = node->next;
      }
      node = next;
    }
    return root;
  }
};
```

49 LC 641 | Design Circular Deque (Medium)

Problem: Implement MyCircularDeque(k), a circular deque of capacity k. insertFront(value), insertLast(value), deleteFront(), and deleteLast() return success; insertions fail when full and deletions when empty. getFront()/getRear() return the end values or -1 when empty. isEmpty()/isFull() report status.
Constraints: 1 <= k <= 1000; 0 <= value <= 1000; At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

```
class MyCircularDeque {
public:
  vector<int> q;
  int front = 0;
  int size = 0;
  int capacity = 0;
  MyCircularDeque(int k) {
    q.assign(k, 0);
    capacity = k;
  }
  bool insertFront(int value) {
    if (isFull()) {
      return false;
    }
    if (!isEmpty()) {
      front = (front - 1 + capacity) % capacity;
    }
    q[front] = value;
    ++size;
    return true;
  }
  bool insertLast(int value) {
    if (isFull()) {
      return false;
    }
    int idx = (front + size) % capacity;
    q[idx] = value;
    ++size;
    return true;
  }
  bool deleteFront() {
    if (isEmpty()) {
      return false;
    }
    front = (front + 1) % capacity;
    --size;
    return true;
  }
  bool deleteLast() {
    if (isEmpty()) {
      return false;
    }
    --size;
    return true;
  }
  int getFront() {
    return isEmpty() ? -1 : q[front];
  }
  int getRear() {
    return isEmpty() ? -1 : q[(front + size - 1) %
      capacity];
  }
  bool isEmpty() {
    return size == 0;
  }
  bool isFull() {
    return size == capacity;
  }
};
```

50 LC 460 | LFU Cache (Hard)

Problem: Implement LFUCache(capacity). get(key) returns its value or -1; put(key,value) inserts or updates. At capacity, evict the least frequently used key, breaking ties by least recent use. New entries have frequency 1; successful gets and updates increment frequency. Both operations require average O(1) time.

Constraints: 1 <= capacity <= 10^4; 0 <= key <= 10^5; 0 <= value <= 10^9; At most 2 * 10^5 calls will be made to get and put.

```
class LFUCache {
  struct Entry { int value, frequency;
    list<int>::iterator position; };
  int capacity, minimum = 0;
  unordered_map<int, Entry> entries;
  unordered_map<int, list<int>> buckets;
  void touch(unordered_map<int, Entry>::iterator
    it) {
    int key = it->first, frequency =
      it->second.frequency;
    buckets.try_emplace(frequency + 1);
    auto& from = buckets.at(frequency);
    auto& to = buckets.at(frequency + 1);
    to.splice(to.begin(), from,
      it->second.position);
    it->second.frequency++;
    it->second.position = to.begin();
    if (from.empty()) {
      buckets.erase(frequency);
      if (minimum == frequency) ++minimum;
    }
  }
public:
  LFUCache(int capacity) : capacity(capacity) {}
  LFUCache(const LFUCache&) = delete;
  LFUCache& operator=(const LFUCache&) = delete;
  int get(int key) {
    auto it = entries.find(key);
    if (it == entries.end()) return -1;
    touch(it);
    return it->second.value;
  }
  void put(int key, int value) {
    if (!capacity) return;
    if (auto it = entries.find(key); it !=
      entries.end()) {
      it->second.value = value;
      touch(it);
      return;
    }
    if (static_cast<int>(entries.size()) ==
      capacity) {
      auto& least = buckets.at(minimum);
      entries.erase(least.back());
      least.pop_back();
      if (least.empty()) buckets.erase(minimum);
    }
    auto& first = buckets[1];
    first.push_front(key);
    entries.emplace(key, Entry{value, 1,
      first.begin()});
    minimum = 1;
  }
};
```